Jobber → Google Calendar Sync — Code Guide (Cloudflare + GitHub)

What the code does, file by file, and how it runs on Cloudflare and ships from GitHub. The full source is in **06-SOURCE-CODE** (appendix) and in `jobber-gcal-sync-source.zip`.

Stack: TypeScript on Cloudflare Workers, with no runtime dependencies (only Web APIs: `fetch`, `crypto.subtle`). Storage is Cloudflare D1 (SQLite) plus Cloudflare Queues. Tests use Vitest; the dev tools are `wrangler`, `typescript` and `vitest`.

---

1. Repository layout

```
jobber-gcal-sync/
├── wrangler.toml              Cloudflare config: bindings, vars, queue, cron
├── package.json               scripts: test, typecheck, dev, deploy, db:migrate:*
├── tsconfig.json · vitest.config.ts
├── migrations/0001_init.sql   D1 schema (5 tables)
├── src/                       the Worker (≈1,900 lines)
├── test/                      65 automated tests + fakes for Jobber/Google/D1
├── graphql/                   the exact GraphQL queries, for checking in Jobber's GraphiQL
├── scripts/                   sign-webhook.mjs (send a signed test webhook), render-docs-pdf.py
└── docs/                      these guides + PDFs
```

`src/`: what each file does

File	Responsibility	Key functions
`index.ts`	**Entry point.** HTTP router (`fetch`), queue consumer (`queue`), daily cron (`scheduled`)	`handleWebhook`, default export `{ fetch, queue, scheduled }`
`env.ts`	Types for every binding, variable and secret, plus the queue message shape	`Env`, `SyncMessage`
`config.ts`	Webhook topic allow-list and routing	`classifyTopic()` → `visit_upsert`, `visit_delete`, `job_create`, `job_update`, `job_delete`, `app_disconnect`, `ignored`
`log.ts`	JSON logs; **redacts** any field that looks like a token/secret/key	`log()`, `snippet()`
`jobber/webhook.ts`	Verifies `X-Jobber-Hmac-SHA256` and parses the payload	`verifyJobberSignature()`, `parseJobberWebhook()`
`jobber/oauth.ts`	Jobber OAuth: authorize redirect (PKCE + state) and callback/code exchange	`startJobberOAuth()`, `handleJobberCallback()`

File	Responsibility	Key functions
`jobber/tokens.ts`	Encrypted token storage; **rotation-safe refresh** with a D1 lock	`getJobberAccessToken()`, `storeTokens()`
`jobber/graphql.ts`	GraphQL client (version header, 401 → refresh, THROTTLED → retry) and the queries	`jobberGraphQL()`, `VISIT_QUERY`, `JOB_VISITS_QUERY`, `USERS_QUERY`, `ACCOUNT_QUERY`
`jobber/ids.ts`	Decodes Jobber EncodedIds (`gid://Jobber/<Type>/<n>`) to reject non-Visit objects	`decodeEncodedId()`, `isDefinitelyNot()`
`google/auth.ts`	Google access token: service-account JWT (RS256) or OAuth refresh; accepts the private key in any paste format	`getGoogleAccessToken()`, `buildServiceAccountJwt()`, `normalizePrivateKey()`
`google/calendar.ts`	Idempotent event **upsert** (PUT → POST → PUT) and **delete** (404/410 = already gone)	`upsertCalendarEvent()`, `deleteCalendarEvent()`
`sync/validate.ts`	The gate before every Google write: Visit? on a Job? scheduled? assigned?	`validateJobVisit()`
`sync/mapEvent.ts`	Jobber visit → Google event body; deterministic event ID; content hash	`buildEventBody()`, `deterministicEventId()`, `contentHash()`
`sync/processor.ts`	**The sync brain:** per topic, decides create/update/move/delete	`processMessage()`, `syncVisit()`, `removeVisit()`
`db/repo.ts`	Every D1 query	accounts, OAuth state, mappings, visit events, sync log
`admin.ts`	JSON admin API (Bearer `ADMIN_API_TOKEN`)	`handleAdmin()`
`adminUi.ts`	Browser pages with a password box: connect Jobber, map team → calendars, recent activity	`handleJobberStartPage()`, `handleAdminPage()`
`util/crypto.ts`	base64/base64url, SHA-256, HMAC, PKCE, constant-time compare, AES-GCM	`encryptString()`, `decryptString()`
`util/retry.ts`	`fetchWithRetry()` (backoff, Retry-After), `RetryableError` vs `PermanentError`, queue backoff	`queueBackoffSeconds()`

2. How a request flows

A. Jobber webhook → calendar (the main path)

```
POST /webhooks/jobber                                      index.ts › handleWebhook
  1. read raw body
  2. verifyJobberSignature(raw, X-Jobber-Hmac-SHA256, JOBBER_CLIENT_SECRET)   → 401 if bad
  3. parseJobberWebhook → {topic, accountId, itemId}
  4. classifyTopic → "ignored"? → 200 and log "Skipped non-job event"
  5. SYNC_QUEUE.send({topic, accountId, itemId})           → 200 "ok" (well under 1 s)

Queue consumer                                             index.ts › queue
  - collapses duplicates in the batch (same kind + account + item)
  - processMessage(msg)                                    sync/processor.ts
      visit_upsert  → syncVisit(visitId)
      visit_delete  → removeVisit(visitId)            (uses stored mapping; no Jobber call)
      job_create    → JOB_VISITS_QUERY → syncVisit for each visit (≤100)
      job_update    → syncVisit for each visit already tracked for that job
      job_delete    → removeVisit for each tracked visit
      app_disconnect→ account status = disconnected
  - success → ack · RetryableError → retry with backoff · PermanentError → log + ack

syncVisit(visitId)
  1. jobberGraphQL(VISIT_QUERY, {id})                      jobber/graphql.ts
  2. validateJobVisit()  → not OK? log reason; remove old events if it had some
  3. look up calendars for assigned users                  db/repo.ts › getCalendarsForUsers
  4. delete events on calendars that no longer apply (reassigned/unmapped)
  5. for each mapped assignee:
       eventId = "jv" + sha256(account|visit|user)[0..40]
       body    = buildEventBody(); hash = contentHash()
       unchanged hash → "noop"
       else save mapping row FIRST → upsertCalendarEvent() → save hash
```

B. Connect Jobber (OAuth)

`GET /oauth/jobber/start` shows a password box (`adminUi.ts`). Submitting the correct `ADMIN_API_TOKEN` stores state plus a PKCE verifier in `oauth_states` and redirects to Jobber. Jobber then calls `/oauth/jobber/callback?code&state` (`oauth.ts`). The Worker checks the state, exchanges the code (with `code_verifier`), queries `account{id name}`, and stores the **encrypted** tokens in `jobber_accounts`.

C. Admin page

`GET /admin` shows the password box. `POST /admin` with the token loads Jobber users (`USERS_QUERY`) and shows a calendar field for each. Saving writes `user_calendar_map`. The page also shows the last 25 `sync_log` rows.

D. Daily cron (`17 4 * * *` UTC)

`repo.prune()` deletes OAuth states older than 1 h and `sync_log` rows older than 30 days.

---

3. Data model (D1, `migrations/0001_init.sql`)

Table	Key	Purpose
`jobber_accounts`	`account_id`	One row per connected Jobber account: AES-GCM-encrypted access/refresh tokens, expiry, refresh lock, status (`active`, `reauth_required`, `disconnected`)
`oauth_states`	`state`	Single-use OAuth state + PKCE verifier (10-min validity)
`user_calendar_map`	`(account_id, jobber_user_id)`	**Jobber user → Google Calendar ID** (platform IDs, not names)
`visit_events`	`(account_id, visit_id, jobber_user_id)`	What we wrote to Google: calendar, event ID, job ID, content hash
`sync_log`	`id`	Audit trail: created / updated / deleted / noop / skipped / error

---

4. Cloudflare: how it's wired

`wrangler.toml`, line by line

Setting	Meaning
`name = "jobber-gcal-sync"`, `main = "src/index.ts"`	Worker name (also the URL prefix) and entry file
`compatibility_date`	Runtime version the Worker is pinned to
`keep_vars = true`	Deploys **keep** variables added in the dashboard (otherwise a Git deploy deletes them)
`[observability] enabled`	Logs appear under Worker → Observability → Events
`[vars]`	Non-secret settings (API version, callback URL, timezone, topics, client ID, service-account email)
`[[d1_databases]] binding="DB"`	Code uses `env.DB`; `database_id` must be **your** D1 ID
`[[queues.producers]] binding="SYNC_QUEUE"`	Code sends with `env.SYNC_QUEUE.send()`
`[[queues.consumers]]`	The same Worker consumes: `max_batch_size=10`, `max_batch_timeout=1` s, `max_retries=8`, **`max_concurrency=1`** (one consumer at a time, so token refreshes never race), `dead_letter_queue`
`[triggers] crons`	Daily cleanup

Secrets are not in this file. They're added in the dashboard (Settings → Variables and Secrets → type **Secret**) or with `npx wrangler secret put NAME`.

Cloudflare dashboard map

Task	Where
See deploys / build logs	Workers & Pages → `jobber-gcal-sync` → **Deployments** (each push = one build)
Live logs, error details	→ **Observability → Events** (expand a row for the JSON fields)
Secrets / variables / bindings	→ **Settings → Variables and Secrets / Bindings**
Git connection, build command	→ **Settings → Build** (repo, branch `main`, deploy command `npx wrangler deploy`)
Run SQL	Storage & Databases → **D1** → `jobber_gcal_sync` → **Console**
Queue depth, DLQ	Compute → **Queues**
workers.dev URL	Worker → **Domains**, or Workers → *Subdomain* (don't change it)

Deploying without GitHub (CLI alternative)

```
npm ci
npx wrangler login
npx wrangler d1 create jobber_gcal_sync            # put the ID in wrangler.toml
npx wrangler d1 execute jobber_gcal_sync --remote --file=migrations/0001_init.sql
npx wrangler queues create jobber-visit-sync
npx wrangler queues create jobber-visit-sync-dlq
npx wrangler secret put JOBBER_CLIENT_SECRET       # repeat for the other 3 secrets
npx wrangler deploy
```

Local development

```
cp .dev.vars.example .dev.vars    # fill in values; never commit
npm run db:migrate:local
npm run dev                       # http://localhost:8787 (local D1 + local queue)
JOBBER_CLIENT_SECRET=... node scripts/sign-webhook.mjs VISIT_UPDATE <visitId> <accountId>
http://localhost:8787/webhooks/jobber
```

---

5. GitHub: how changes ship

Piece	Role
Repo (private), branch `main`	Source of truth. `main` = **production**: every push to `main` triggers a Cloudflare build and deploy (~1 min)

Piece	Role
Cloudflare Workers and Pages GitHub app	Lets Cloudflare clone the repo for builds. It must be authorized for the repo owner's account
Claude GitHub app (optional)	Lets a Claude agent push commits. Install with *Only select repositories* → *this repo*

Changing code from the GitHub website (no terminal):

1. Open the file on github.com → the pencil **Edit** icon → change it → **Commit changes** to `main`.
2. Cloudflare → Deployments: a new build appears; wait for **Ready**.
3. Check `/health` and Observability → Events.

For risky changes, commit to a branch and open a pull request. Merge into `main` only after running `npm test` (locally or in a Codespace). Pushes to `main` go live immediately.

Rolling back: Cloudflare → Deployments → pick an earlier version → **Rollback**, or revert the commit on GitHub.

Never commit: `.dev.vars`, the Google JSON key, or any token. `.gitignore` already excludes `.dev.vars`, `.wrangler/` and `node_modules/`.

---

6. Common changes: where to edit

I want to…	Edit
Change the event title format or add a prefix	`EVENT_TITLE_PREFIX` var, or `buildEventBody()` in `src/sync/mapEvent.ts`
Add more info to the event description	`buildEventBody()` (description array) + add the field to `VISIT_QUERY` in `src/jobber/graphql.ts`
Change the timezone for all-day visits	`DEFAULT_TIMEZONE` in `wrangler.toml`
React to another Jobber webhook topic	Add it to `ALLOWED_TOPICS` + `DEFAULT_ALLOWED_TOPICS` (`src/config.ts`), handle its kind in `processMessage()`, subscribe it in the Jobber app
Stop showing events as busy	`transparency` in `buildEventBody()` (`"transparent"`)
Upgrade the Jobber API version	`JOBBER_API_VERSION` in `wrangler.toml` (check the Jobber changelog first)
Change retry behaviour	`src/util/retry.ts` and `max_retries` in `wrangler.toml`

After any change: `npm run typecheck && npm test`, then push.

---

7. Tests

`npm test` runs **65 tests** in about 2 seconds, with no network or credentials. `test/helpers/` contains:

- `d1.ts` : a real SQLite database built from the actual migration file
- `fakes.ts` : in-memory **Jobber** (OAuth + GraphQL, with refresh-token rotation) and **Google Calendar** (keeps cancelled events like Google does), served through a stubbed `fetch`

Covered: create, update, delete, reassign, two assignees, duplicates, unscheduled, Requests/Quotes/Assessments never synced, `JOB_CREATE`, all-day events, retries and throttling, token refresh and rotation, invalid refresh token, `APP_DISCONNECT`, no tokens in logs, HMAC, PKCE, the service-account JWT, every private-key paste format, and the admin pages.